

Module 6: Day 1 Command Reference Card

Quick reference for all command types and options covered today.

General Options (Available on All Commands)

Option	Type	Default	Description
type	str	(required)	Command type (shell, ssh, regex, etc.)
cmd	str	(required)	Command content (meaning varies by type)
save	str	-	Save output to a file path
metadata	dict	-	Key-value pairs logged but not executed
exit_on_error	bool	True	Stop playbook on non-zero return code
error_if	regex	-	Raise error if pattern matches output
error_if_not	regex	-	Raise error if pattern does NOT match output
only_if	condition	-	Execute only if condition is true
loop_if	regex	-	Retry if pattern matches output
loop_if_not	regex	-	Retry if pattern does NOT match output
loop_count	int	3	Max retry iterations
background	bool	False	Run as background process
kill_on_exit	bool	True	Kill background process when playbook ends

Command Types

shell:Local Command Execution

```
- type: shell
  cmd: nmap -T4 $TARGET
  # Optional:
  command_shell: /bin/bash      # Default: /bin/sh
  interactive: False           # Interactive mode
  creates_session: "name"      # Create named session
  session: "name"              # Reuse session
  command_timeout: 15          # Timeout for interactive mode
```

ssh:Remote Command Execution

```
- type: ssh
  cmd: id
  hostname: $TARGET
  username: user
  password: user
  # Optional:
  port: 22
  key_filename: /path/to/key
  timeout: 60
  creates_session: "name"
  session: "name"
  interactive: False
  command_timeout: 15
  prompts: ["$ ", "# ", "> "]
  clear_cache: False
```

sftp:File Transfer

```
- type: sftp
  cmd: put                                # or "get"
  local_path: /local/file
  remote_path: /remote/file
  # Optional:
  mode: "755"                            # File permissions (upload only)
  session: "name"                        # Reuse SSH session
  hostname: $TARGET                      # Or use cached settings
  username: user
  password: user
```

regex:Pattern Matching

```
- type: regex
  cmd: "pattern with (groups)"
  output:
    VARNAME: "$MATCH_0"
  # Optional:
  mode: findall                          # findall, search, split, sub
  input: RESULT_STDOUT                  # Variable name (without $)
  replace: "replacement"               # For mode: sub
```

debug:Inspect and Troubleshoot

```
- type: debug
  cmd: "Message with $VARIABLES"
  # Optional:
  varstore: False                       # Dump all variables
```

```
wait_for_key: False      # Pause for Enter
exit: False              # Stop playbook
```

setvar:Set a Variable

```
- type: setvar
  variable: VARNAME      # Without $
  cmd: "value with $SUBSTITUTION"
  # Optional:
  encoder: base64-encoder # base64-encoder/decoder, rot13,
                           urlencoder/decoder
```

mktemp:Create Temporary File/Directory

```
- type: mktemp
  variable: TEMPFILE     # Without $
  cmd: file               # "file" or "dir"
```

sleep:Pause Execution

```
- type: sleep
  seconds: 5
  # Optional:
  random: False
  min_sec: 0              # Lower bound for random sleep
```

loop:Iterate

```
- type: loop
  cmd: "items(LIST_VAR)"  # or "range(0, 10)" or "until($X == done)"
  commands:
    - type: shell
      cmd: echo $LOOP_ITEM # $LOOP_ITEM for items(), $LOOP_INDEX for
range()
  # Optional:
  break_if: "$DONE == yes"
```

include:Include Another Playbook

```
- type: include
  local_path: path/to/other_playbook.yml
```

Builtin Variables

Variable	Set By	Description
<code>\$RESULT_STDOUT</code>	Most commands	Output of the last command
<code>\$RESULT_RETURNCODE</code>	Most commands	Return code of the last command
<code>\$MATCH_0, \$MATCH_1, ...</code>	regex	Captured groups from regex
<code>\$REGEX_MATCHES_LIST</code>	regex	All matches as a list
<code>\$LOOP_ITEM</code>	loop (items)	Current list element
<code>\$LOOP_INDEX</code>	loop (range/until)	Current iteration index

Conditional Operators

Operator	Example	Description
<code>==</code>	<code>\$PORT == 80</code>	Equal
<code>!=</code>	<code>\$CODE != 0</code>	Not equal
<code>>, >=, <, <=</code>	<code>\$COUNT > 1</code>	Numeric ordering
<code>=~</code>	<code>\$OUT =~ .*open.*</code>	Regex match
<code>!~</code>	<code>\$OUT !~ .*closed.*</code>	Regex no match
<code>is</code>	<code>\$VAR is None</code>	Identity check
<code>is not</code>	<code>\$VAR is not None</code>	Negated identity
<code>not</code>	<code>not \$FLAG</code>	Negation